

Lecture Notes: RMS→RMS Norms, μP , and the Muon Optimizer

Compiled from handwritten lecture notes

December 8, 2025

Contents

1	Key concepts and terms (quick glossary)	2
2	Lecture 6: RMS→RMS norms and why they show up	2
2.1	Induced RMS→RMS operator norm	2
2.2	Motivation (Xavier-style thinking)	3
3	The optimizer recipe as a norm-constrained problem	3
3.1	General template	3
3.2	Example: spectral-norm constrained update yields $UV^\top$	3
4	Using RMS→RMS in the recipe gives layer-wise effective step sizes	3
5	Batch size 1, rank-1 gradients, and signSGD scaling	4
5.1	Why the per-layer gradient is rank 1 (batch size 1)	4
5.2	Rank-1 fact: spectral norm equals Frobenius norm	4
5.3	Applying the RMS→RMS constraint to signSGD	4
6	Maximal Update Parameterization (μP): intuition and desiderata	4
6.1	A scalar warm-up: choosing the “right units”	4
6.2	Reparameterization idea	5
6.3	Feature-learning desiderata for a linear layer	5
7	Lecture 7: from Shampoo to Muon	5
7.1	Shampoo (matrix preconditioning)	5
7.2	A revealing special case: no accumulation	5
7.3	Why compute $UV^\top$?	6
8	Muon: Momentum Orthogonalized by Newton–Schulz	6
8.1	High-level algorithm	6
8.2	Newton–Schulz as “polynomial on singular values”	6
8.3	Why normalization is needed	6
8.4	Practical polynomials (NanoGPT example)	7

1 Key concepts and terms (quick glossary)

- **Vector RMS norm.** For $x \in \mathbb{R}^d$,

$$\|x\|_{\text{RMS}} := \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2} = \frac{1}{\sqrt{d}} \|x\|_2.$$

- **Induced/operator norm.** For a matrix A and vector norms $\|\cdot\|_\alpha, \|\cdot\|_\beta$,

$$\|A\|_{\alpha \rightarrow \beta} := \max_{x \neq 0} \frac{\|Ax\|_\beta}{\|x\|_\alpha} = \max_{\|x\|_\alpha=1} \|Ax\|_\beta.$$

- **Spectral norm.** $\|A\|_2 = \sigma_{\max}(A) = \max_{\|x\|_2=1} \|Ax\|_2$.

- **Frobenius norm.** $\|A\|_F = \sqrt{\sum_{ij} a_{ij}^2} = \sqrt{\sum_k \sigma_k(A)^2} = \sqrt{\text{tr}(A^\top A)}$.

- **Semi-orthogonal matrix.** A rectangular matrix $Q \in \mathbb{R}^{m \times n}$ is *semi-orthogonal* if $QQ^\top = I_m$ (when $m \leq n$) or $Q^\top Q = I_n$ (when $n \leq m$). Such matrices have condition number 1 on their nonzero singular values.

- **Optimizer recipe (constrained linearization).** Choose a norm to constrain the update ΔW , then solve a local linear objective $\min_{\|\Delta W\| \leq \eta} \langle \nabla_W \mathcal{L}(W), \Delta W \rangle$.

- **signSGD.** $W_{t+1} = W_t - \eta \text{sgn}(g_t)$, where $g_t = \nabla_W \mathcal{L}(W_t)$ and $\text{sgn}(\cdot)$ is applied elementwise.

- **Maximal Update Parameterization (μP).** A scaling/parameterization philosophy that aims to make activations and *updates* have stable ($\Theta(1)$) magnitudes as width changes, so hyperparameters transfer across model widths.

- **Shampoo.** A preconditioned optimizer using matrix inverse fractional powers of accumulated gradient covariance (left and right).

- **Muon.** *Momentum Orthogonalized by Newton–Schulz*: uses momentum plus an efficient approximate “semi-orthogonalization” (replacing singular values by ≈ 1) via Newton–Schulz polynomial iterations, avoiding explicit SVD.

2 Lecture 6: RMS→RMS norms and why they show up

2.1 Induced RMS→RMS operator norm

Let $A \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ map $x \in \mathbb{R}^{d_{\text{in}}}$ to $Ax \in \mathbb{R}^{d_{\text{out}}}$. Define the induced RMS→RMS norm:

$$\|A\|_{\text{RMS} \rightarrow \text{RMS}} := \max_{\|x\|_{\text{RMS}}=1} \|Ax\|_{\text{RMS}}.$$

Because $\|x\|_{\text{RMS}} = \|x\|_2 / \sqrt{d_{\text{in}}}$ and $\|Ax\|_{\text{RMS}} = \|Ax\|_2 / \sqrt{d_{\text{out}}}$,

$$\begin{aligned} \|A\|_{\text{RMS} \rightarrow \text{RMS}} &= \max_{\|x\|_2 = \sqrt{d_{\text{in}}}} \frac{1}{\sqrt{d_{\text{out}}}} \|Ax\|_2 \\ &= \sqrt{\frac{d_{\text{in}}}{d_{\text{out}}}} \max_{\|u\|_2=1} \|Au\|_2 = \sqrt{\frac{d_{\text{in}}}{d_{\text{out}}}} \|A\|_2. \end{aligned}$$

Takeaway: the induced RMS→RMS norm is a *dimension-rescaled spectral norm*.

2.2 Motivation (Xavier-style thinking)

A common motivation for RMS scaling is that in many random-initialization regimes, keeping RMS magnitudes stable across layers is easier to reason about than keeping raw ℓ_2 norms stable, because fan-in/fan-out appear explicitly through the $\sqrt{d_{\text{in}}/d_{\text{out}}}$ factor.

3 The optimizer recipe as a norm-constrained problem

3.1 General template

Given current parameters W , consider a first-order approximation:

$$\mathcal{L}(W + \Delta W) \approx \mathcal{L}(W) + \langle \nabla_W \mathcal{L}(W), \Delta W \rangle.$$

To choose an update, pick a step-size budget η and a norm $\|\cdot\|$, then solve

$$\Delta W^* \in \arg \min_{\|\Delta W\| \leq \eta} \langle \nabla_W \mathcal{L}(W), \Delta W \rangle.$$

Interpretation: “Choose a geometry (norm), then move as far as allowed in the steepest descent direction under that geometry.”

3.2 Example: spectral-norm constrained update yields $UV^\top$

Let $G = \nabla_W \mathcal{L}(W) \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ with SVD $G = U\Sigma V^\top$. If we constrain $\|\Delta W\|_2 \leq \eta$ (spectral norm), the optimizer of the linear objective is

$$\Delta W^* = -\eta UV^\top,$$

a **semi-orthogonal update** (singular values all equal to 1 on the support of G).

4 Using RMS→RMS in the recipe gives layer-wise effective step sizes

Suppose we constrain the *RMS→RMS* operator norm of the update:

$$\|\Delta W\|_{\text{RMS} \rightarrow \text{RMS}} \leq \gamma.$$

Using $\|\Delta W\|_{\text{RMS} \rightarrow \text{RMS}} = \sqrt{\frac{d_{\text{in}}}{d_{\text{out}}}} \|\Delta W\|_2$, this is equivalent to

$$\|\Delta W\|_2 \leq \gamma \sqrt{\frac{d_{\text{out}}}{d_{\text{in}}}}.$$

Key observation: with a single global hyperparameter γ , different layers (different fan-in/out) effectively have different spectral-norm budgets, hence *layer-specific effective learning rates*. This is one route toward width-aware scaling rules.

5 Batch size 1, rank-1 gradients, and signSGD scaling

5.1 Why the per-layer gradient is rank 1 (batch size 1)

For a linear layer $h = Wx$ with loss $\ell(h)$, the gradient is

$$\nabla_W \ell = \frac{\partial \ell}{\partial h} x^\top,$$

an outer product of a d_{out} vector and a d_{in} vector, hence **rank 1**. This is the “low-rank gradient” intuition used in the notes.

5.2 Rank-1 fact: spectral norm equals Frobenius norm

If A has rank 1 with singular values $\{\sigma, 0, 0, \dots\}$, then

$$\|A\|_2 = \sigma = \|A\|_F.$$

5.3 Applying the RMS→RMS constraint to signSGD

Let $S = \text{sgn}(G)$ be the elementwise sign of the gradient. In the batch-size-1 rank-1 setting, the notes argue S is also rank 1 (up to sign-pattern structure), so $\|S\|_2 = \|S\|_F$. Since entries of S are ± 1 , we have

$$\|S\|_F^2 = \sum_{i=1}^{d_{\text{out}}} \sum_{j=1}^{d_{\text{in}}} 1 = d_{\text{out}} d_{\text{in}} \quad \Rightarrow \quad \|S\|_2 = \sqrt{d_{\text{out}} d_{\text{in}}}.$$

With signSGD update $\Delta W = \eta S$,

$$\|\Delta W\|_{\text{RMS} \rightarrow \text{RMS}} = \eta \sqrt{\frac{d_{\text{in}}}{d_{\text{out}}}} \|S\|_2 = \eta \sqrt{\frac{d_{\text{in}}}{d_{\text{out}}}} \sqrt{d_{\text{out}} d_{\text{in}}} = \eta d_{\text{in}}.$$

To keep $\|\Delta W\|_{\text{RMS} \rightarrow \text{RMS}} \leq \gamma$, it suffices to choose

$$\boxed{\eta \leq \frac{\gamma}{d_{\text{in}}}}$$

so the learning rate scales like $1/d_{\text{in}}$.

Conservativeness beyond batch size 1. As batch size increases, gradients become higher-rank. In general $\|A\|_2 \leq \|A\|_F$, so replacing the spectral norm by Frobenius norm yields an *upper bound* and can be conservative. The notes remark that improved approximations to semi-orthogonal updates can reduce this conservativeness.

6 Maximal Update Parameterization (μP): intuition and desiderata

6.1 A scalar warm-up: choosing the “right units”

Consider i.i.d. $z_1, \dots, z_n$ with mean 0 and variance 1. Then

$$\frac{1}{\sqrt{n}} \sum_{i=1}^n z_i \Rightarrow \mathcal{N}(0, 1),$$

while $\frac{1}{n} \sum z_i \rightarrow 0$ and $\sum z_i$ blows up in scale. If an objective depends on a scaled sum, choosing the scaling factor (here $1/\sqrt{n}$) can make the large- n behavior nontrivial and stable.

6.2 Reparameterization idea

If you have an objective $F_n(c) = \mathbb{E}[f(c(z_1 + \dots + z_n))]$, a reparameterization $c = \alpha/\sqrt{n}$ can yield a stable limit:

$$G_n(\alpha) := F_n(\alpha/\sqrt{n}) \rightarrow \mathbb{E}[f(\alpha \cdot \mathcal{N}(0, 1))],$$

so the optimal α^* transfers across n (at least approximately), whereas the optimal c_n^* may not.

6.3 Feature-learning desiderata for a linear layer

For layer ℓ (ignoring bias and nonlinearity), $h_\ell = W_\ell h_{\ell-1}$ with $W_\ell \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$. Two desiderata emphasized in the notes are:

$$(D1) \quad \|h_\ell\|_{\text{RMS}} = \Theta(1), \quad (D2) \quad \|\Delta h_\ell\|_{\text{RMS}} = \Theta(1) \text{ per step.}$$

A convenient sufficient condition is to keep both the weights and their updates controlled in RMS→RMS norm:

$$\boxed{\|W_\ell\|_{\text{RMS} \rightarrow \text{RMS}} = \Theta(1), \quad \|\Delta W_\ell\|_{\text{RMS} \rightarrow \text{RMS}} = \Theta(1).}$$

Sketch of why: using $\Delta h_\ell = \Delta W_\ell h_{\ell-1} + W_\ell \Delta h_{\ell-1}$ and submultiplicativity,

$$\|\Delta h_\ell\|_2 \leq \|\Delta W_\ell\|_2 \|h_{\ell-1}\|_2 + \|W_\ell\|_2 \|\Delta h_{\ell-1}\|_2,$$

and converting between ℓ_2 and RMS introduces explicit fan-in/out scaling; controlling RMS→RMS norms keeps the recursion stable across widths.

7 Lecture 7: from Shampoo to Muon

7.1 Shampoo (matrix preconditioning)

Let $G_t = \nabla_W \mathcal{L}(W_t)$. Shampoo maintains left/right accumulators

$$L_t = L_{t-1} + G_t G_t^\top, \quad R_t = R_{t-1} + G_t^\top G_t,$$

and applies the update

$$W_{t+1} = W_t - \eta L_t^{-1/4} G_t R_t^{-1/4}.$$

Intuition: the preconditioner reduces anisotropy so steps are more “uniform” across directions.

7.2 A revealing special case: no accumulation

If we ignore accumulation and use $L = G G^\top$ and $R = G^\top G$, then

$$\Delta W \propto (G G^\top)^{-1/4} G (G^\top G)^{-1/4}.$$

With SVD $G = U \Sigma V^\top$ (assume full rank for simplicity),

$$G G^\top = U \Sigma^2 U^\top, \quad G^\top G = V \Sigma^2 V^\top,$$

so

$$(G G^\top)^{-1/4} G (G^\top G)^{-1/4} = U \Sigma^{-1/2} U^\top \cdot U \Sigma V^\top \cdot V \Sigma^{-1/2} V^\top = U V^\top.$$

So “Shampoo without accumulation” produces the same **semi-orthogonal direction** $U V^\top$ that arises from a spectral-norm constrained step.

7.3 Why compute $UV^\top$?

Replacing Σ by the identity removes domination by the largest singular value, leading to condition number 1 (on supported subspace) and a more uniform step across directions.

8 Muon: Momentum Orthogonalized by Newton–Schulz

8.1 High-level algorithm

Muon maintains a momentum-like buffer and then approximately “orthogonalizes” it.

Algorithm 1 Muon (conceptual form)

Require: learning rate η , momentum $\mu \in [0, 1)$, iteration count K

```

1: initialize  $B_0 \leftarrow 0$ 
2: for  $t = 1, 2, \dots$  do
3:    $G_t \leftarrow \nabla_W \mathcal{L}(W_t)$ 
4:    $B_t \leftarrow \mu B_{t-1} + G_t$ 
5:    $O_t \leftarrow \text{NEWTONSCHULZORTHOGONALIZE}(B_t; K)$   $\triangleright O_t \approx UV^\top$ 
6:    $W_{t+1} \leftarrow W_t - \eta O_t$ 
7: end for
```

8.2 Newton–Schulz as “polynomial on singular values”

A key trick: odd polynomials of the form

$$p(X) = a_0 X + a_1 X(X^\top X) + a_2 X(X^\top X)^2 + \dots$$

commute with the SVD. If $X = U\Sigma V^\top$, then

$$p(X) = U p(\Sigma) V^\top,$$

so applying p changes singular values but preserves singular vectors.

A classic Newton–Schulz polynomial for pushing singular values toward 1 (for $\sigma \in (0, 1)$) is

$$p(\sigma) = \frac{3}{2}\sigma - \frac{1}{2}\sigma^3,$$

leading to the matrix iteration

$$X_{k+1} = \frac{3}{2}X_k - \frac{1}{2}X_k X_k^\top X_k.$$

Iterating drives Σ_k toward I (approximately), hence $X_k \approx UV^\top$.

8.3 Why normalization is needed

The iteration is stable when singular values lie in a suitable range (heuristically $(0, 1)$ in the notes).

A simple way to enforce this is

$$X_0 \leftarrow \frac{B}{\|B\|_F},$$

because $\|B\|_2 \leq \|B\|_F$, so all singular values of X_0 are ≤ 1 .

8.4 Practical polynomials (NanoGPT example)

The notes mention using a higher-order odd polynomial that better matches the “spectrum” encountered in practice (e.g. NanoGPT):

$$f(\sigma) = 3.4442 \sigma - 4.7750 \sigma^3 + 2.0315 \sigma^5.$$

This may sacrifice strict convergence guarantees for speed/quality of approximation; the goal is a good-enough semi-orthogonal direction.